Minimum spanning trees

Part VI Graph Algorithms

Scope

Safe edges across cuts reduce the cost of connectivity. Exchange arguments establish Kruskal and Prim. One five-vertex graph tracks components, keys, and parents, leading to independently checkable certificates of optimality and uniqueness.

Prerequisites

Connected undirected graphs, cuts, and weights.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Derive MST safety from the cut property.
- Compare Kruskal and Prim, including their structures and costs.
- Identify assumptions about negative weights, disconnected graphs, and uniqueness.
- 23.1 Growing a minimum spanning tree
- 23.2 The algorithms of Kruskal and Prim

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

23.1 Growing a minimum spanning tree

Minimize total tree weight, not each path length

Let G=(V,E) be a finite undirected graph, with n=|V|,m=|E| and comparable finite real edge weights w(e). First assume n≥1 and G connected. A spanning tree T contains all vertices, is connected and acyclic, and has exactly n−1 edges. An MST minimizes $w(T) = \sum_{e \in T} w(e)$. There are finitely many candidate trees, so a minimum exists. Weights may be negative, zero, or repeated.

Negative weights do not invalidate MSTs, but acyclicity remains essential. In a triangle with all weights−1, a tree costs−2, whereas the general connected subgraph using all edges costs−3. An MST need not be a shortest-path tree. For ab:4,ac:5,bc:2, the MST {ab,bc} costs 6, and its a-to-c path has length6; the shortest graph path ac has length5.

For n=1, return no edges and cost 0; for n=0, use the empty-forest convention. A disconnected graph has no spanning tree. Its minimum spanning forest, MSF, consists of an MST in each original component. Self-loops belong to no tree. Parallel edges retain distinct edge IDs, so endpoints alone do not identify a removable edge.

One five-vertex graph, two growth orders

Use the following fixed input throughout. Kruskal breaks weight ties by edge ID.

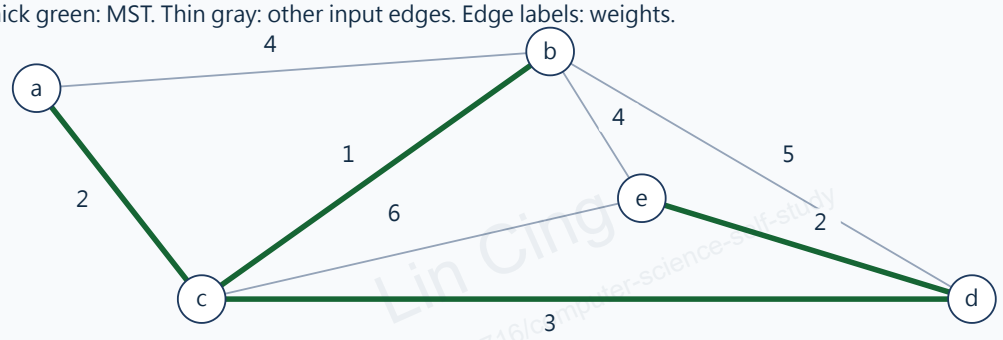
ID	0	1	2	3	4	5	6	7
Edge	ab	ac	bc	bd	cd	ce	de	be
Weight	4	2	1	5	3	6	2	4

Kruskal selects bc,ac,de,cd; Prim from a selects ac,cb,cd,de. Both end with {bc,ac,de,cd}, costing1+2+2+3=8, but their intermediate forests and grown vertex sets differ. Gray edges also belong to the input. The final green tree alone does not determine the selection process.

One five-vertex graph, two growth orders (continued)

Complete input and a minimum spanning tree

Thick green: MST. Thin gray: other input edges. Edge labels: weights.



Choose bc1,ac2,de2,cd3: 4 edges connect 5 vertices, cost 8.

Safe means extendible to an optimal tree

Maintain that $A \subseteq E$ is contained in at least one MST. An edge $e \notin A$ is safe for A if some MST contains $A \cup \{e\}$. This optimal-extension requirement is stronger than adding e without a cycle, which only preserves a forest.

A cut $(S, V-S)$ requires $\varnothing \neq S \neq V$. An edge crosses it when its endpoints lie on opposite sides. The cut respects A if no A edge crosses it. A light edge has minimum weight among the crossing edges, possibly tied.

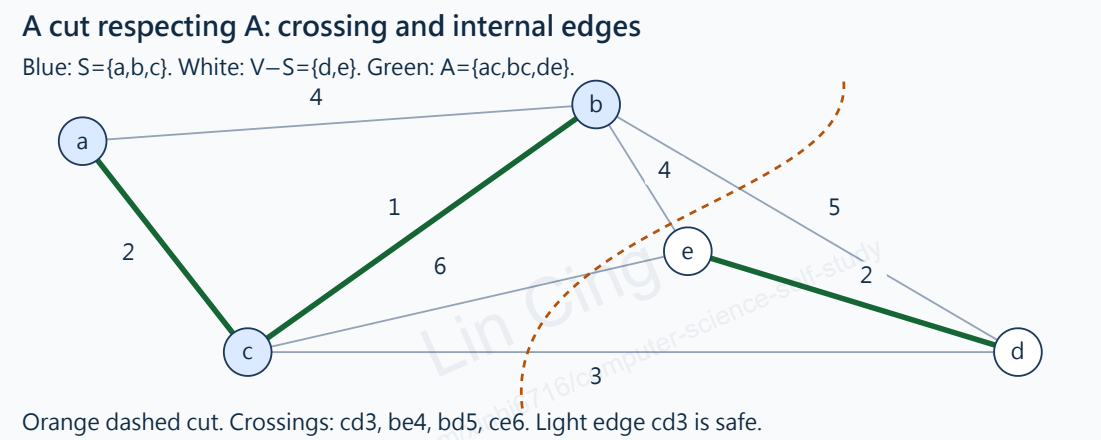
The generic algorithm starts with $A=\varnothing$ and repeatedly adds a safe edge until $|A|=n-1$. The invariant ensures acyclicity. A forest on n vertices with $n-1$ edges is connected, so the result is an MST. The algorithmic task is to establish safety efficiently; choosing a cheap edge is not itself a proof.

Safe-edge theorem and preservation of selected edges

Suppose A extends to an MST, cut (S,V−S) respects A, and e is a light edge of the cut. Choose an MST T containing A. If e∈T, the claim already holds. Otherwise T+e has exactly one cycle. Since e crosses the cut, the cycle must cross it again at some edge f to return to its starting side. Since the cut respects A, f∉A.

Removing f breaks the cycle while preserving connectivity, so $T' = T - \{f\} + \{e\}$ is a spanning tree. Because $w(e) \leq w(f)$, its cost is at most w(T). Optimality of T makes T′ an MST containing $A \cup \{e\}$. The exchange proves structural validity, nonincreasing cost, and preservation of A.

Safe-edge theorem and preservation of selected edges (continued)



Here S={a,b,c},A={ac,bc,de}. Crossing edges are cd:3,be:4,bd:5,ce:6, so cd is safe. Edge de lies entirely on the other side.

Quantifiers for ties, forced edges, and cycles

If e is the **unique** light edge of a cut, an MST omitting it could be strictly improved by the exchange. Thus every MST contains e. A tied light edge need only belong to some MST. A triangle with all weights 1 has three MSTs; every edge is light for some cut, but all three cannot be selected together.

Cycle property: a uniquely heaviest edge e on a cycle belongs to no MST. If an MST T contained e, removing it would define a cut. The remaining original cycle path supplies a crossing edge f with w(f)<w(e), giving a strict improvement. If e is only tied for heaviest, a nonincreasing exchange proves that some MST omits it, not that every MST must omit it.

Every component C of forest A defines a respecting cut (C,V−C). Its lightest outgoing edge is therefore safe. Kruskal and Prim share this argument; neither requires selecting the globally cheapest edge at every iteration.

23.2 The algorithms of Kruskal and Prim

Kruskal merges components in global weight order

```
KRUSKAL(V,E):
    A=empty set
    for v in V: MAKE-SET(v)
    sort E nondecreasingly by (weight,edge-ID)
    for e=(u,v) in sorted E:
        ru=FIND(u); rv=FIND(v)
        if ru!=rv:
            A.add(e); UNION(ru,rv)
    return A
```

The DSU represents current components of (V,A), not final components of the original graph. Equal roots mean an existing A path, so adding e would cycle; this also rejects self-loops. Distinct roots join two trees with one edge and preserve acyclicity. For connected input, stop after accepting n−1 edges. For an MSF, scanning all input edges is also valid.

Kruskal's correctness and earlier cheap edges

Assume A extends to an MST before the next accepted edge e=(u,v). Let S be u's current DSU component, whose cut respects A. Any crossing edge f preceding e has already been processed. If accepted then, it joined its endpoints permanently, so it cannot cross the current S. If rejected then, its endpoints were already in one component and remain together, also impossible for a crossing edge. Thus no earlier edge crosses this cut.

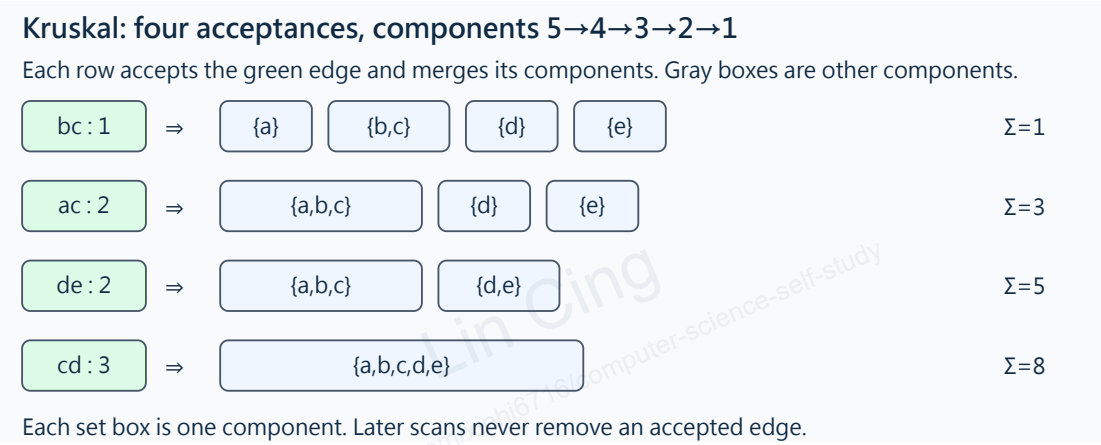
Unscanned edges have weight at least w(e), making e light for the cut. The safe-edge theorem preserves optimal extendibility. At termination, if an original component still consisted of multiple A components, an original connecting path would contain an edge crossing A components. That edge could not have been rejected when scanned, a contradiction. Thus every original component receives an optimal spanning tree, and the output is an MSF.

The proof requires neither distinct weights nor minimum-vertex DSU representatives. Root identities are implementation details.

Every Kruskal state

Scanned edge	Endpoint components before scan	Decision	Components afterward	Cumulative cost
bc:1	{b},{c}	accept	{bc},{a},{d},{e}	1
ac:2	{a},{bc}	accept	{abc},{d},{e}	3
de:2	{d},{e}	accept	{abc},{de}	5
cd:3	{abc},{de}	accept	{abcde}	8
ab:4	same component	reject	unchanged	8
be:4	same component	reject	unchanged	8
bd:5	same component	reject	unchanged	8
ce:6	same component	reject	unchanged	8

Every Kruskal state (continued)



Step 4 already accepts $n-1=4$ edges and permits stopping. The final four rows check the full-scan version. At step 3, de need not touch {abc}: Kruskal grows a forest rather than Prim's single tree.

Kruskal's cost and graph assumptions

Comparison sorting costs $O(m \log(m+1))$. Rank-and-compression DSU handles n MAKE-SET calls,$2m$ FIND calls, and at most $n-1$ UNION calls in $O((n+m)(1+\alpha(n)))$. Under the Chapter 21 convention $\alpha(2)=0$, so only $n \geq 3$ permits the simplification $O((n+m)\alpha(n))$; retain the constant term for smaller n. Including input gives $O(m \log(m+1)+(n+m)(1+\alpha(n)))$. An empty graph returns an empty forest directly. The sorted edge list uses $O(m)$ space, DSU $O(n)$, and output $O(n)$.

For a connected **simple graph**, $n-1 \leq m < n^2$, so $\log m = O(\log n)$, yielding the familiar $O(m \log n)$. Arbitrarily many parallel edges invalidate $m<n^2$; retain $\log m$ unless an additional algorithm removes redundant edges. Negative and equal weights do not affect comparison sorting.

For integer weights $0..W$, counting sort can replace comparison sorting in $O(m+W)$, but DSU cost remains. A large W can make this worse. This changes the representation assumption behind the comparison lower bound.

Prim's key is one crossing edge, not path distance

For the grown set S, every v∉S maintains $key[v] = \min_{u \in S, (u,v) \in E} w(u, v)$, or∞ when no such edge exists. Its parent records the endpoint in S and edge ID realizing the value. Each unselected vertex has one priority-queue item. Set the first root's key to 0 only to extract it first; this zero is not an output edge weight.

Extract a minimum key each iteration. Except at the initial root, its parent edge is a light edge across (S,V−S) and is safe. After extracting u, compare each new candidate w(u,v) directly with key[v], not key[u]+w(u,v).

For ab:4,ac:5,bc:2, extracting a gives keys b4,c5. Extracting b lowers c's key to 2, although the tree path from a costs 4+2=6. Dijkstra compares d[b]+2=6 and retains d[c]=5. Prim permits negative weights; Dijkstra's correctness has its own nonnegative-edge assumption.

Prim's complete queue contract

```
PRIM-CONNECTED(V,Adj,r):
    for v in V: key[v]=∞; parent[v]=NIL; inQ[v]=true
    key[r]=0; Q=BUILD-INDEXED-MIN-HEAP(V,key)
    A=empty set
    while Q is not empty:
        u=EXTRACT-MIN(Q); inQ[u]=false
        if key[u]==∞: report disconnected input
        if parent[u]!=NIL: A.add(the edge recorded in parent[u])
        for each edge e=(u,v) in Adj[u]:
            if inQ[v] and w(e)<key[v]:
                parent[v]=(u,e.ID)
                DECREASE-KEY(Q,v,w(e)); key[v]=w(e)
    return A
```

DECREASE-KEY synchronizes heap and key storage. Array inQ gives O(1) membership; scanning the heap for membership changes the cost. For an MSF, extracting∞ starts a new original component: make u a root, set key[u]=0, add no parent edge, and scan normally. Do not add∞ to the forest cost.

Prim's invariant, termination, and complete trace

At each iteration start S=V−Q; A is a tree on S, empty before the first iteration, contained in an MST. Each queued vertex's key and parent specify its lightest edge to S. Initialization handles the root separately. Extraction adds a safe edge by the cut theorem, and scanning u supplies exactly all newly available crossing candidates. Thus the invariant is restored. Each iteration removes one vertex, so n iterations produce an n−1-edge MST.

Extracted item	Remaining key(parent) values after scan	Accepted edges	Cost
a:0	b4(a),c2(a),d∞,e∞	none	0
c:2	b1(c),d3(c),e6(c)	ac	2
b:1	d3(c),e4(b)	ac,cb	3
d:3	e2(d)	ac,cb,cd	6
e:2	empty	ac,cb,cd,de	8

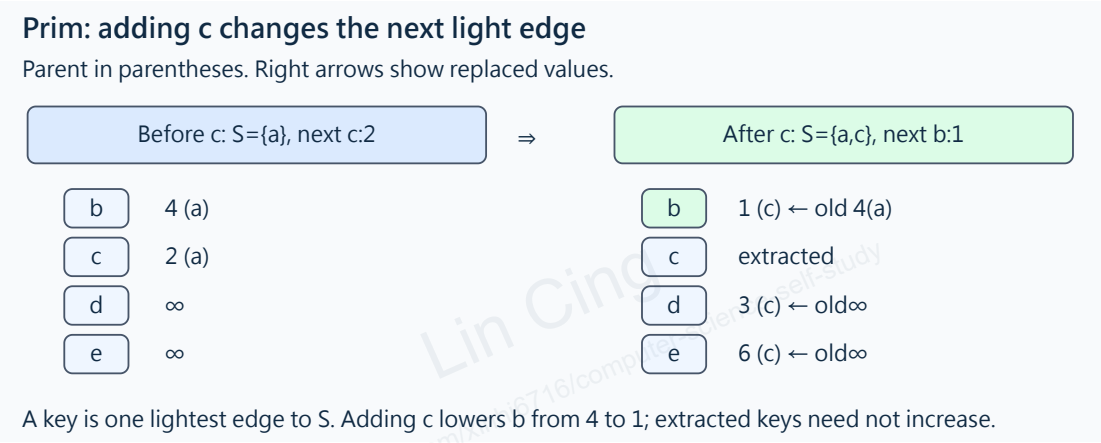
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prim's invariant, termination, and complete trace (continued)

Extracted keys0,2,1,3,2 are not nondecreasing. A new vertex can provide a cheaper crossing edge. Optimality follows from cuts, rather than key monotonicity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

New candidates determine the next safe edge



For S={a}, ac:2 is lightest. Extracting c lowers b from ab:4 to cb:1 and gives d,e their first finite keys. The next extraction must be b:1; having just extracted2 cannot exclude weight 1. Extracting b replaces ce:6 by be:4 for e, and extracting d later replaces that by de:2.

New candidates determine the next safe edge (continued)

Save every parent change together with the corresponding key. The final parent need not be the first encountered neighbor. Vertex extraction order alone cannot reconstruct these competing candidates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Keep Prim's implementation costs distinct

Queue and representation	Work	Total cost
Indexed binary heap and adjacency lists	n extracts, at most2m scanned entries and m decreases	O((n+m)log n)
Fibonacci heap and adjacency lists	O(n log n) amortized extraction, O(m) amortized decreases	O(m+n log n)
Linear minimum-key search and adjacency lists	n scans of at most n keys,2m edge entries	$O(n^2+m)$
Adjacency matrix and linear keys	n rounds scanning n keys and n neighbor positions	$\Theta(n^2)$

For a simple graph, $m=O(n^2)$, making the third row $O(n^2)$. Handle n=1 directly with an empty tree. Indexed handles locate a decrease in O(1). A lazy heap that pushes replacement entries and discards stale ones can contain O(m) items; the n-item proof does not apply unchanged.

Keep Prim's implementation costs distinct (continued)

For $m=\Theta(n)$, both heap variants give O(n log n). For $m=\Theta(n^2)$, binary and Fibonacci bounds are respectively $O(n^2 \log n)$ and $O(n^2)$, and linear-key scanning also gives $O(n^2)$. The executable example verifies choices using dense Prim; it does not claim a measured Fibonacci-heap speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Certify a candidate tree without rerunning the algorithm

Let T be a spanning tree. Quantify only over **non-self-loop nontree edges** e=(u,v),u≠v, and let M_T(u,v) be the maximum weight on the unique T path from u to v. Ignore self-loops; no maximum is assigned to an empty u-to-u path. Then

$$T \text{ is an MST} \quad \Longleftrightarrow \quad \forall e = (u,v) \in E \setminus T,\ u \neq v:\ w(e) \geq M_T(u,v).$$

Necessity follows by adding any e with w(e)<M and removing a maximum path edge, strictly improving T. For sufficiency, remove any f∈T to define a cut. Every other crossing edge e is outside T and its T path uses f, giving w(e)≥M_T≥w(f). Thus f is light across that cut. No other T edge crosses it, so the cut respects any previously selected prefix of T. Add T's edges in any order and repeatedly apply the safe-edge theorem to prove optimality.

For nontree edges ab,bd,ce,be in the fixed graph, pairs (w,M) are (4,2),(5,3),(6,3),(4,3). All inequalities are strict, proving uniqueness here.

Uniqueness and changing one edge weight

Tree T is unique exactly when every non-self-loop nontree edge has w(e)>M_T. Equality allows replacing a maximum path edge with e at equal cost. Conversely, strictness makes f the unique light edge of every T−f cut. Every MST must then contain every edge of T. This does not require **all input weights to differ**: the example has two weight 2 edges and a unique MST.

Lower only one non-self-loop nontree edge e to w′, keeping everything else fixed, and let M be its original T-path maximum. If w′≥M, T still passes every certificate. If w′<M, replace a maximum path edge f by e. The new optimum is w(T)+min(0,w′−M). For a lower bound, a tree omitting e retains original weight at least w(T). For any new tree R containing e, remove e to define a cut. The original T path between its endpoints contains a crossing edge g. Tree R−e+g exists in the original weighted graph, hence w(T)≤w(R)−w′+w(g)≤w(R)−w′+M. This matches the constructed bound.

Uniqueness and changing one edge weight (continued)

Lowering be from 4 to 1 uses path b-c-d-e with weights 1,3,2, so remove cd:3 and get8−3+1=6. Lowering only to 3 leaves the old tree optimal but creates a tie. A certificate using a separate DFS per path costs O(nm). This update guarantee concerns one changed edge. No self-loop, however negative, can enter a tree; changing its weight leaves the MST unchanged and never triggers a tree-path exchange.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Uniqueness and changing one edge weight (continued)

Nontree edges and path maxima: certificates and updates

The unique original tree path from b to e. Nontree edge be is shown below.



Orange cd:3 is the path maximum M=3

Original be:4 > 3 No improvement	Lower be to 3 Same-cost exchange	Lower be to 1 Remove cd, add be

Original cost 8 → tie stays 8 → at weight 1: 8−3+1=6.

Keep ac:2 and the other tree edges. One cycle exchange leaves exactly 4 edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 23.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study